PRIMERA PARTE:

Test, 10 o 15 minutos, 3 preguntas, en papel, sobre lo que hemos hecho en prácticas

SEGUNDA PARTE: EXPERIMENTO 💀:

Práctica 1

- En sklearn, para qué sirve el parámetro `k` (`n_components`) en PCA?
- De qué va mnist (números), fashion_mnist (prendas de ropa), cifar-10 (imágenes de varias clases)? (No hay que saber la resolución de las imágenes)
- Saber cómo funciona PCA
- Diferencia entre los métodos `fit(X)`, `transform(X)`, `fit_transform(X)`, `inverse_transform(X)`
  - Si llamamos a `fit(X)`, luego tenemos que llamar a transform(X)
  - `fit_transform(X)` hace las 2 cosas
  - `inverse_transform(X)` pasa los datos pequeñitos de nuevo a la dimensión original
- Qué es el error de reconstrucción?

Práctica 2

- Qué es:
  - `reg_param` (interpola la matriz de varianzas con la matriz identidad) (es un parámetro de regularización)
  - `tol` (tolerancia) en LDA suaviza varianzas y covarianzas
- Saber utilizar LDA y QDA
- Búsqueda de hiperparámetros (puede que haya que usarlo en el problema):

```
clf = LinearDiscriminantAnalysis(); G = {'tol': loguniform(1e-9, 1e-1)}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
RS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=split
```

Práctica 3: Regresión logística

- `tol` (tolerancia) (umbral de error de error a partir del cual si bajamos de ahí, paramos el entrenamiento, porque consideramos que el modelo ya es suficientemente bueno)
- Ingeniería de características:
  - `degree` es el grado máximo de las características generadas
  - `interaction_only` (boolean): Si nos cargamos lo original para quedarnos con las polinomial features

Posibles ejercicios:

Para el clasificador QDA, haz una búsqueda de hiperparámetros para el parámetro reg_param (práctica 2):

- Forma 1:

```
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.model_selection import train_test_split, ShuffleSplit, RandomizedSearchCV
from scipy.stats import loguniform; import time; start = time.time()
clf = QuadraticDiscriminantAnalysis(); G = {'reg_param': loguniform(1e-6, 1e-1)}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
RS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=8)
acc = RS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.2%} con {RS.best_params_['tol']:.4f}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))
```

- Forma 2:

```
for p in [1e-6, 1e-5, 1e-4, 1e-3, 1e-2, 1e-1]:
  clf = QuadraticDiscriminantAnalysis(reg_param=p)
  acc = clf.fit(X_train, y_train).score(X_test, y_test)

--NORMAL--
```

En regresión logística, haz una búsqueda de hiperparámetros para el parámetro tol (práctica 3):

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, ShuffleSplit, RandomizedSearchCV
from scipy.stats import loguniform; import time; start = time.time()
clf = LogisticRegression(); G = {'tol': loguniform(1e-6, 1e-1)}
splitter = ShuffleSplit(n_splits=2, test_size=0.1, random_state=23)
RS = RandomizedSearchCV(clf, G, n_iter=10, scoring='accuracy', n_jobs=-1, cv=splitter, pre_dispatch=8)
acc = RS.fit(X_train, y_train).score(X_test, y_test)
print(f"Precisión: {acc:.2%} con {RS.best_params_['tol']:.4f}")
print('Tiempo (hh:mm:ss):', time.strftime('%H:%M:%S', time.gmtime(time.time() - start)))

--NORMAL--                                                    <Esc>
```